

Operating System

Rapid-Revision One-Liners

IBPS SO (IT OFFICER)
COMPLETE PREPARATION SERIES

100% SYLLABUS COVERAGE
CONCEPT-FOCUSED LEARNING
MCQ-ORIENTED APPROACH
QUICK REVISION READY

COMPILED BY
Asabhya Maanav

2026 EDITION
VERSION 1.0

Table of Contents

Operating System — One-Liners	4
Introduction and OS Basics	4
OS Structures and Operating Modes	4
Types of Operating Systems	4
Process Concepts and PCB	5
Process Scheduling and Schedulers	5
Threads and Multithreading	5
Interprocess Communication (IPC)	6
CPU Scheduling Criteria and Basics	6
CPU Scheduling Algorithms	6
Process Synchronization Basics	7
Semaphores	7
Classical Synchronization Problems	7
Monitors and High-Level Constructs	7
Deadlock Concepts	8
Deadlock Handling and Banker's Algorithm	8
Memory Management Basics	8
Contiguous Allocation and Fragmentation	8
Paging	9
Segmentation	9
Virtual Memory and Demand Paging	9
Page-Replacement Algorithms	9
Thrashing and Working Set	10
File Management	10
File Allocation and Free-Space Management	10
Protection and Access Control	11
I/O Systems	11
Disk Structure and Scheduling	11
RAID and Disk Management	11
Bootting, Interrupts and Kernel Internals	11
Advanced Process and Thread Concepts	12
More CPU Scheduling Facts	12
Advanced Synchronization	12
More Deadlock Facts	12

Advanced Memory Management	12
More Virtual Memory Facts	13
Advanced File Systems	13
Advanced Disk and I/O	13
High-Yield Numbers and Difference Recap	13
Specialized Scheduling and IPC	14
System Software: Loaders, Linkers, Compilers	14
Page-Table Entry and TLB Details	14
OS History and Examples	14
More Numerics, Sizes and Edge Cases	15

Operating System — One-Liners

High-density, double-layered rapid-revision facts for the **IBPS SO (IT Officer)** exam. Each line pairs a structural metric (how many / which / what size / what complexity) with the descriptive answer.

Introduction and OS Basics

1. An **Operating System** is **system software** acting as an intermediary between **user/application programs** and **computer hardware**.
2. The **2** primary goals of an OS are **convenience** (user goal) and **efficiency** (system goal); the secondary goal is **ability to evolve**.
3. An OS has **2** classic views: **Resource Manager** (bottom-up: allocates CPU, memory, I/O) and **Extended/Virtual Machine** (top-down: hides hardware details).
4. The **always-resident** core of an OS is the **kernel**, loaded at power-on by the **bootstrap loader** stored in **ROM/firmware**.
5. The **8** core OS functions are **process, memory, file, I/O-device, secondary-storage management**, plus **protection/security, networking, and command interpretation**.
6. A **system call** is the interface to OS services and triggers a **mode switch** from **user mode to kernel mode** via a **software interrupt (trap)**.
7. There are **6** categories of system calls: **process control, file management, device management, information maintenance, communication, and protection**.
8. **Process-control** system calls include **fork()**, **exec()**, **exit()**, and **wait()**.
9. **File-management** system calls include **open()**, **read()**, **write()**, and **close()**.
10. **fork()** returns **0** to the child and the **child's PID** to the parent, while **exec()** replaces the process image without creating a new process.
11. System-call parameters are passed by **3** methods: via **registers**, via a **memory block/table**, or via the **stack**.
12. **System programs** sit between the kernel and applications and fall in **6** classes: **file management, status info, file modification, programming-language support, program loading/execution, and communications**.
13. The **shell** is the **command interpreter**; common examples are **Bash, PowerShell**, and the **C-shell (csh)**.

OS Structures and Operating Modes

14. The **5** main OS architectures are **monolithic, layered, microkernel, modular, and hybrid**.
15. A **microkernel** keeps only **IPC, scheduling, and low-level memory** in the kernel; examples are **Mach, QNX, L4, and MINIX**.
16. **Layered** OS uses **N** layers where **layer 0** is the **hardware** and each layer uses services of the one below it.
17. Mostly **monolithic** Oses include **Linux and UNIX**, while **hybrid** Oses include **Windows NT and macOS (XNU)**.
18. Dual-mode operation uses **1** mode bit: **0 = kernel/supervisor mode** (all instructions) and **1 = user mode** (no privileged instructions).
19. Intel CPUs provide **4** protection rings where **Ring 0 = kernel** and **Ring 3 = user**.
20. A **privileged instruction** (I/O, halt, set timer) run in user mode causes a **trap** to the OS.
21. There are **2** hypervisor types: **Type-1 (bare-metal)** like **VMware ESXi, Xen, Hyper-V** and **Type-2 (hosted)** like **VirtualBox, VMware Workstation**.

Types of Operating Systems

22. **Simple batch systems** use a **resident monitor** and have **no CPU-I/O overlap**, leaving the CPU **idle during I/O**.
23. **Multiprogramming** keeps **many jobs** in memory to maximize **CPU utilization** by switching on **I/O waits**.
24. **Time-sharing/multitasking** adds **rapid switching** for **interactive** users, targeting **response time** under about **1 second**.
25. Multiprocessor systems are of **2** kinds: **Symmetric (SMP)** where each CPU runs the same OS, and **Asymmetric (AMP)** with a **master-slave** relationship.
26. **Real-time systems** are of **2** types: **hard** (guaranteed deadlines: airbags, pacemakers) and **soft** (priority, tolerable misses: streaming).

27. A **distributed OS** makes many machines look like **1** system (transparency), whereas a **network OS (NOS)** keeps machines **visibly separate**.
28. **Clustered systems** link **independent nodes** for **high availability** in **2** modes: **asymmetric** (hot-standby) and **symmetric**.
29. Common **embedded/handheld** OSes include **Android, iOS, FreeRTOS, and VxWorks**, designed for **limited memory/CPU/power**.
30. Multiprocessor systems give **3** benefits: **increased throughput, economy of scale, and increased reliability (graceful degradation)**.

Process Concepts and PCB

31. A **program** is a **passive** disk entity while a **process** is an **active** program in execution with PC, registers, and stack.
32. A process address space has **4** sections: **text (code), data (globals), heap (dynamic, grows up), and stack (grows down)**.
33. A process has **5** states: **new, ready, running, waiting/blocked, and terminated**.
34. On an **I/O request** a process goes **running --> waiting**, while on **preemption/timeout** it goes **running --> ready**.
35. A **waiting** process always returns to **ready** (never directly to **running**) when its event completes.
36. The **Process Control Block (PCB)**, also called **Task Control Block**, stores **all** information about a process.
37. The PCB holds **8** key fields: **process state, PID, program counter, CPU registers, scheduling info, memory-management info, accounting info, and I/O status**.
38. The collection of all **PCBs** maintained by the OS is the **process table**.
39. Memory-management info in the PCB includes **base/limit registers, page tables, and segment tables**.
40. The number of processes in memory is the **degree of multiprogramming**, controlled by the **long-term scheduler**.

Process Scheduling and Schedulers

41. The OS maintains **3** scheduling queues: the **job queue, the ready queue, and device/I/O queues**.
42. There are **3** schedulers: **long-term (job), short-term (CPU), and medium-term (swapper)**.
43. The **short-term scheduler** runs **most frequently** (milliseconds) and must be **fast**.
44. The **long-term scheduler** runs **least frequently** and controls the **degree of multiprogramming**.
45. The **medium-term scheduler** performs **swapping** of processes in and out of memory.
46. A **context switch** saves/restores the **PCB** and is **pure overhead**, reduced by hardware **multiple register sets**.
47. The **dispatcher** gives CPU control to the chosen process and performs **context switch, switch to user mode, and jump** to the program.
48. **Dispatch latency** is the time the dispatcher takes to **stop one process and start another**.
49. A good process mix balances **CPU-bound** and **I/O-bound** processes to keep both CPU and devices busy.

Threads and Multithreading

50. A **thread** is the **basic unit of CPU utilization**, sharing **code, data, and files** but owning its **PC, registers, and stack**.
51. Threads provide **4** benefits: **responsiveness, resource sharing, economy, and scalability**.
52. Threads are of **2** kinds: **user-level threads (ULT)** managed by a library, and **kernel-level threads (KLT)** managed by the OS.
53. In **user-level threads**, a single **blocking call blocks the whole process** and there is **no true multiprocessor parallelism**.
54. There are **3** multithreading models: **Many-to-One, One-to-One, and Many-to-Many**.
55. **One-to-One** (Windows, Linux) gives **true parallelism** but has **per-thread overhead**, while **Many-to-One** is cheap but offers **no parallelism**.
56. Compared to a process, a thread is **lightweight**, has **faster context switch**, and **shares** an address space.
57. A thread crash **can kill the whole process**, whereas a process crash **does not** kill other processes.

Interprocess Communication (IPC)

58. IPC uses **2** fundamental models: **shared memory** (fast, programmer-synchronized) and **message passing** (kernel-mediated, slower).
59. **Message passing** uses **2** primitives: **send()** and **receive()**.
60. Communication links are **2** types: **direct** (name sender/receiver) and **indirect** (via **mailboxes/ports**).
61. Send/receive can be **synchronous (blocking)** or **asynchronous (non-blocking)**.
62. Message buffering has **3** capacities: **zero (rendezvous)**, **bounded**, and **unbounded**.
63. The canonical shared-memory cooperation example is the **Producer-Consumer (Bounded-Buffer)** problem.
64. **Pipes** are of **2** kinds: **ordinary pipes** (related processes) and **named pipes/FIFOs** (unrelated processes).
65. A **socket** is a communication endpoint identified by an **IP address + port number**.
66. **Signals** are **asynchronous** notifications such as **SIGKILL**, **SIGINT**, and **SIGSEGV**.
67. Shared memory involves the kernel **only at setup**, while message passing involves the kernel for **every message**.

CPU Scheduling Criteria and Basics

68. Process execution alternates between a **CPU burst** and an **I/O burst** (the **CPU-I/O burst cycle**).
69. CPU-burst durations follow an **exponential/hyperexponential** distribution: **many short, few long** bursts.
70. There are **5** scheduling criteria: **CPU utilization**, **throughput**, **turnaround time**, **waiting time**, and **response time**.
71. **CPU utilization** and **throughput** are **maximized**, while **turnaround**, **waiting**, and **response time** are **minimized**.
72. **Turnaround Time = Completion Time - Arrival Time**.
73. **Waiting Time = Turnaround Time - Burst Time**.
74. **Response Time = first CPU response - arrival time** (counts only until the **first** response).
75. **Throughput** is the number of processes **completed per unit time**.
76. Scheduling can occur at **4** points; using only **(running-to-waiting)** and **(termination)** gives **non-preemptive** scheduling.
77. **Preemptive** scheduling can forcibly remove the CPU; examples are **Round-Robin**, **SRTF**, and **preemptive Priority**.
78. **Non-preemptive** scheduling keeps the CPU until completion or wait; examples are **FCFS**, **SJF**, and **non-preemptive Priority**.

CPU Scheduling Algorithms

79. **FCFS** uses a **FIFO queue**, is **non-preemptive**, and suffers the **convoy effect**.
80. **SJF** picks the **smallest next CPU burst** and is **provably optimal** for **minimum average waiting time**.
81. **SJF** has **2** variants: **non-preemptive SJF** and **preemptive SJF = SRTF (Shortest Remaining Time First)**.
82. SJF needs the **next burst length**, predicted by **exponential averaging**: $\tau(n+1) = \alpha * t(n) + (1 - \alpha) * \tau(n)$.
83. **Priority scheduling** gives the CPU to the **highest priority**, usually a **smaller integer = higher priority**.
84. Priority scheduling can cause **starvation (indefinite blocking)**, cured by **aging** (raising priority over time).
85. **Round-Robin** uses a fixed **time quantum (q)** and is **preemptive** and **starvation-free**.
86. In Round-Robin, **q --> infinity** behaves like **FCFS**, while a **very small q** causes high **context-switch overhead**.
87. A Round-Robin rule of thumb: **80%** of CPU bursts should be **shorter than the quantum**.
88. With **n** processes and quantum **q**, no process waits more than **(n - 1) * q** for its next turn.
89. **Multilevel Queue (MLQ)** permanently assigns processes to queues with **no movement** between them.
90. **Multilevel Feedback Queue (MLFQ)** allows processes to **move between queues** via **promotion (aging)** and **demotion**.
91. **MLFQ** is defined by **5** parameters: number of queues, per-queue algorithm, promotion rule, demotion rule, and entry queue.
92. **HRRN** is **non-preemptive** and selects the highest **Response Ratio = (Waiting Time + Burst Time) / Burst Time**.
93. **HRRN** avoids **starvation** because a long-waiting job's response ratio keeps **increasing**.
94. For FCFS with bursts **P1=24, P2=3, P3=3** (all at t=0), the **average waiting time** is **17 ms**.
95. For non-preemptive SJF with bursts **6, 8, 7, 3**, the **average waiting time** is **7 ms**.

- 96. For the classic SRTF set, the **average waiting time** is **6.5 ms** and **average turnaround** is **13 ms**.
- 97. For Round-Robin with bursts **24, 3, 3** and **q = 4**, the **average waiting time** is about **5.66 ms**.
- 98. The **5** common scheduling algorithms causing starvation are **SJF, SRTF, Priority, MLQ (low queues); RR and HRRN** do not starve.
- 99. **Multiprocessor scheduling** uses **processor affinity** (soft or hard) and **load balancing** via **push** and **pull migration**.
- 100. **Real-time scheduling** has **2** classics: **Rate-Monotonic (static, shorter period = higher priority)** and **EDF (dynamic, earliest deadline)**.
- 101. **Rate-Monotonic** schedulability bound for many tasks approaches **In 2 ~ 69.3%**, while **EDF** can reach **100%** utilization.
- 102. Thread contention scope is **2** types: **PCS (process-contention scope)** and **SCS (system-contention scope)**.

Process Synchronization Basics

- 103. A **race condition** occurs when the result depends on the **order** of concurrent access to **shared data**.
- 104. A correct critical-section solution must satisfy **3** requirements: **Mutual Exclusion, Progress, and Bounded Waiting**.
- 105. The critical-section code has **4** parts: **entry section, critical section, exit section, and remainder section**.
- 106. **Peterson's solution** is a **2-process** software solution using **flag[2]** and **turn**, satisfying all **3** requirements.
- 107. **Dekker's algorithm** was the **first** correct software solution to the **2-process** mutual-exclusion problem.
- 108. Hardware synchronization support includes **3** approaches: **disabling interrupts, Test-and-Set, and Compare-and-Swap/Swap**.
- 109. **Disabling interrupts** works only on **uniprocessors** and fails on **multiprocessors**.
- 110. **Test-and-Set** and **Compare-and-Swap** are **atomic** instructions guaranteeing **mutual exclusion** but **not bounded waiting** alone.

Semaphores

- 111. A **semaphore** is an integer accessed by **2** atomic operations: **wait()/P()** and **signal()/V()**, invented by **Dijkstra**.
- 112. **P** stands for **proberen (to test)** and **V** stands for **verhogen (to increment)**.
- 113. Semaphores are **2** types: **counting** (unrestricted range, N instances) and **binary** (value **0 or 1**, = **mutex**).
- 114. A **counting semaphore** is initialized to the number of **available resource instances**.
- 115. The simple semaphore definition causes **busy-waiting (spinlock)**, wasting CPU but avoiding context switches.
- 116. The block-wakeup implementation lets a semaphore go **negative**, where **|value| = number of blocked processes**.
- 117. A **binary semaphore (mutex)** initialized to **1** provides **mutual exclusion** for a critical section.

Classical Synchronization Problems

- 118. There are **4** classical synchronization problems: **Bounded-Buffer, Readers-Writers, Dining Philosophers, and Sleeping Barber**.
- 119. The **Bounded-Buffer** problem uses **3** semaphores: **mutex = 1, empty = n, and full = 0**.
- 120. In the producer, the correct order is **wait(empty) then wait(mutex)**; reversing them causes **deadlock**.
- 121. The **Readers-Writers** problem has **2** variants: **first** (readers priority, writers may starve) and **second** (writers priority, readers may starve).
- 122. The **Dining Philosophers** problem uses **5** philosophers and **5** forks, each needing **2** forks to eat.
- 123. Dining Philosophers deadlocks if **all 5** pick the **left fork** at once; fixes include allowing **at most 4** at the table or **asymmetric** pickup.
- 124. The **Sleeping Barber** problem has **1** barber, **1** barber chair, and **N** waiting chairs.

Monitors and High-Level Constructs

- 125. A **monitor** is an **abstract data type** allowing **only 1** process active inside it at a time (automatic mutual exclusion).
- 126. A monitor uses **condition variables** with **2** operations: **wait()** and **signal()**.

127. A **monitor's signal()** on an empty condition is **lost**, whereas a **semaphore's signal()** is **remembered (counted)**.

128. Monitor signalling has **2** disciplines: **signal-and-wait** and **signal-and-continue**.

129. A **conditional critical region** uses the form **region v when (B) S;** with automatic mutual exclusion.

Deadlock Concepts

130. A **deadlock** is a set of processes each waiting for an event that **only another process in the set** can cause.

131. The resource-use protocol has **3** steps: **request**, **use**, and **release**.

132. There are **4** necessary (Coffman) conditions: **Mutual Exclusion**, **Hold and Wait**, **No Preemption**, and **Circular Wait**.

133. **All 4** Coffman conditions must hold **simultaneously** for deadlock; breaking **any 1** prevents it.

134. A **Resource-Allocation Graph (RAG)** uses **2** edge types: **request edge (process to resource)** and **assignment edge (resource to process)**.

135. In a RAG, **no cycle** means **no deadlock**; a cycle with **single-instance** resources means **deadlock**.

136. With **multiple-instance** resources, a RAG cycle is **necessary but not sufficient** for deadlock.

Deadlock Handling and Banker's Algorithm

137. There are **4** deadlock-handling strategies: **prevention**, **avoidance**, **detection and recovery**, and **ignorance (Ostrich algorithm)**.

138. **UNIX and Windows** use the **Ostrich algorithm** (ignore deadlocks) for everyday operation.

139. **Deadlock prevention** negates **1** of the **4** conditions; negating **circular wait** uses a **total ordering** of resources.

140. Negating **hold and wait** forces requesting **all resources at once**, causing **low utilization and starvation**.

141. **Deadlock avoidance** uses **a-priori maximum needs** to keep the system in a **safe state**.

142. A **safe state** guarantees **no deadlock**, but an **unsafe state** only means deadlock is **possible**.

143. Every **deadlock state is unsafe**, but **not every unsafe state is a deadlock**.

144. The **Banker's algorithm** handles **multiple instances** using **4** structures: **Available**, **Max**, **Allocation**, and **Need**.

145. In Banker's, **Need = Max - Allocation**.

146. The Banker's **safety algorithm** finds a **safe sequence** by repeatedly choosing a process whose **Need ≤ Work**.

147. For the classic Banker's example with Available **(3,3,2)**, a safe sequence is **P1, P3, P4, P0, P2**.

148. The **resource-request algorithm** grants a request only if **Request ≤ Need**, **Request ≤ Available**, and the result stays **safe**.

149. **Deadlock detection** with single instances uses a **wait-for graph** where a **cycle = deadlock**, costing **O(n²)**.

150. **Deadlock recovery** uses **2** methods: **process termination** and **resource preemption (with rollback)**.

151. In **deadlock no process progresses**, but in **starvation other processes do progress** while one waits indefinitely.

Memory Management Basics

152. A **logical (virtual) address** is generated by the **CPU**; a **physical address** is seen by the **memory unit (RAM)**.

153. The **Memory-Management Unit (MMU)** maps logical to physical addresses at **run time** using the **relocation/base register**.

154. **Address binding** can occur at **3** times: **compile time**, **load time**, and **execution time**.

155. Only **execution-time** binding allows a process to **move in memory during execution** and needs the **MMU**.

156. **Dynamic loading** loads a routine **only when called**, improving memory use without special OS support.

157. **Dynamic linking** postpones library linking to **run time** using a **stub**, sharing **1** copy of a DLL/.so.

158. **Swapping** moves a process to a **backing store (disk)** and back, increasing the **degree of multiprogramming**.

Contiguous Allocation and Fragmentation

159. Contiguous partitioning is **2** types: **fixed (static)** causing **internal** fragmentation and **variable (dynamic)** causing **external** fragmentation.

160. There are **4** dynamic allocation strategies: **First Fit**, **Best Fit**, **Worst Fit**, and **Next Fit**.

- 161. **First Fit** is the **fastest**; **Best Fit** leaves the **smallest leftover** but causes the **most tiny holes** and is **slowest**.
- 162. **Worst Fit** allocates the **largest hole** and performs **worst** in practice.
- 163. **Internal fragmentation** is wasted space **inside** an allocation; **external fragmentation** is **scattered free space** outside.
- 164. **Paging causes internal** fragmentation; **segmentation/variable partitioning causes external** fragmentation.
- 165. **Compaction** cures **external** (not internal) fragmentation and requires **dynamic relocation**.
- 166. The **50-percent rule** says first-fit statistically loses about **0.5N** blocks to fragmentation for **N** allocated blocks.

Paging

- 167. **Paging** divides physical memory into **frames** and logical memory into equal-size **pages** (typically **4 KB**, a power of 2).
- 168. **Paging eliminates external fragmentation** but causes **internal fragmentation** in the **last page**.
- 169. A paged logical address has **2** parts: **page number (p)** and **page offset (d)**.
- 170. For address space 2^m and page size 2^n , the page number uses $(m - n)$ bits and the offset uses n bits.
- 171. The physical address is computed as **frame number * frame size + offset**.
- 172. The **TLB (Translation Lookaside Buffer)** is a small **associative cache** of recent page-table entries.
- 173. **Effective Access Time** with TLB = $h * (TLB + mem) + (1 - h) * (TLB + 2 * mem)$, where **h = hit ratio**.
- 174. For TLB = **20 ns**, memory = **100 ns**, and hit ratio = **0.80**, the EAT is **140 ns**.
- 175. **Multilevel paging** pages the page table itself, increasing translation **memory accesses**.
- 176. An **inverted page table** has **1 entry per physical frame**, sized to **physical** memory, but is **slower** to search.
- 177. **Shared pages** allow re-entrant **read-only** code (e.g., an editor) to be shared by mapping the **same frames**.

Segmentation

- 178. **Segmentation** supports the **user view** as **variable-size logical segments** like **code, stack, and heap**.
- 179. A segmented address has **2** parts: **segment number (s)** and **offset (d)**.
- 180. The **segment table** stores **2** values per segment: a **base** and a **limit**; the MMU checks **offset < limit**.
- 181. **Segmentation with paging** pages each segment to remove external fragmentation, as in **Intel x86** and **MULTICS**.
- 182. **Paging** uses **fixed** blocks and is **invisible** to the programmer; **segmentation** uses **variable** blocks and is **visible**.
- 183. **Segmentation** makes **sharing and protection easier** because they apply to a whole **logical segment**.

Virtual Memory and Demand Paging

- 184. **Virtual memory** lets a process run while **not fully in memory**, so a program can be **larger than physical RAM**.
- 185. Virtual memory gives **3** benefits: **larger programs**, **more processes in memory**, and **less I/O** for loading/swapping.
- 186. **Demand paging** loads a page **only when referenced**, using a **lazy swapper (pager)**.
- 187. A page-table **valid-invalid bit** marks whether a page is **in memory** or **not**.
- 188. **Pure demand paging** starts a process with **zero** pages, faulting in **every** page on first use.
- 189. **Pre-paging** loads pages **expected** soon to reduce the number of **page faults**.
- 190. **Page-fault handling** takes **6** steps: check validity, trap if invalid, find a free frame, schedule disk read, update page table, and **restart the instruction**.
- 191. Demand-paging $EAT = (1 - p) * \text{memory access} + p * \text{page-fault service time}$, where **p = page-fault rate**.
- 192. With memory = **200 ns** and page-fault service = **8 ms**, a fault rate of **1/1000** gives EAT about **8200 ns**.
- 193. Disk access (**milliseconds**) is roughly **100,000 times** slower than RAM (**nanoseconds**), so **p** must be tiny.
- 194. The **modify (dirty) bit** avoids writing back an **unmodified** victim page during replacement.

Page-Replacement Algorithms

- 195. The **6** main page-replacement algorithms are **FIFO**, **Optimal (MIN)**, **LRU**, **Second-Chance/Clock**, **LFU**, and **MFU**.
- 196. **FIFO** replaces the **oldest** loaded page and is the algorithm that suffers **Belady's anomaly**.

197. **Belady's anomaly** means **increasing frames can increase page faults**, demonstrated by FIFO on string **1 2 3 4 1 2 5 1 2 3 4 5**.
198. For that string, FIFO gives **9 faults with 3 frames** but **10 faults with 4 frames** (the anomaly).
199. **Optimal (MIN)** replaces the page **not used for the longest future time**, giving the **fewest** faults but being **unrealizable**.
200. **LRU** replaces the page **not used for the longest past time** and is a **stack algorithm** (no Belady's anomaly).
201. For the reference string **7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1** with **3 frames**, FIFO = **15**, LRU = **12**, and OPT = **9** faults.
202. **LRU** can be implemented with **counters/timestamps** or a **stack** of page numbers.
203. **Second-Chance (Clock)** uses a **reference bit**: a bit of **1** gets a second chance (cleared), and a bit of **0** is replaced.
204. **Enhanced second-chance** uses the **(reference, modify)** pair to prefer evicting **unmodified** pages.
205. **LFU** replaces the page with the **smallest reference count**; **MFU** replaces the one with the **largest count**.
206. **Stack algorithms** like **LRU** and **OPT** never suffer **Belady's anomaly** due to the **subset (inclusion) property**.

Thrashing and Working Set

207. **Thrashing** is high paging activity where a process spends **more time paging than executing**, collapsing **CPU utilization**.
208. Thrashing is caused by **too high a degree of multiprogramming**, giving each process **too few frames**.
209. Frame allocation is **2** schemes: **equal allocation (m/n)** and **proportional allocation (by size or priority)**.
210. Replacement scope is **2** types: **global** (take any process's frame) and **local** (only own frames).
211. The **working set** is the set of pages referenced in the last **Delta** references (the working-set window).
212. If total working-set demand **D > total frames m**, the system **thrashes**.
213. The **Page-Fault Frequency (PFF)** scheme controls thrashing **directly** using **upper and lower** fault-rate bounds.
214. Thrashing is reduced by **decreasing** the degree of multiprogramming, **local replacement**, and the **working-set/PFF** models.

File Management

215. A **file** is a **named collection of related information** on secondary storage.
216. A file has key **attributes** including **name, identifier, type, location, size, protection, and timestamps**.
217. There are **6** basic file operations: **create, write, read, reposition (seek), delete, and truncate**.
218. File **access methods** are **3** types: **sequential, direct/relative, and indexed**.
219. **Sequential access** suits **tapes and logs**; **direct access** suits **databases**.
220. **Directory structures** are **5** types: **single-level, two-level, tree, acyclic-graph, and general-graph**.
221. An **acyclic-graph** directory allows **sharing but no cycles**; a **general-graph** allows **cycles** but needs **garbage collection**.
222. A **tree** directory does **not** allow **shared files/links**, unlike acyclic and general graphs.

File Allocation and Free-Space Management

223. File-allocation methods are **3** types: **contiguous, linked (FAT), and indexed**.
224. **Contiguous allocation** gives **fast access** but causes **external fragmentation** and hard **file growth**.
225. **Linked allocation** avoids external fragmentation but gives **poor direct access**; its table form is the **FAT (used by MS-DOS)**.
226. **Indexed allocation** uses an **index block** of pointers, supporting **direct access** without external fragmentation.
227. The **UNIX inode** uses a combined scheme: **12 direct + 1 single-indirect + 1 double-indirect + 1 triple-indirect** pointers.
228. Free-space management uses **4** techniques: **bit vector, linked list, grouping, and counting**.
229. A **bit vector (bitmap)** uses **1 bit per block** where **1 = free** and **0 = used**.
230. The **counting** method stores an **address + count** of contiguous free blocks.

Protection and Access Control

- 231. The **access matrix** has **rows = domains** and **columns = objects**, with entries listing **access rights**.
- 232. An **Access Control List (ACL)** is a **column** of the matrix (per object); a **capability list** is a **row** (per domain).
- 233. UNIX permissions use **3** classes (**owner, group, others**) and **3** rights: **read = 4, write = 2, execute = 1**.
- 234. **chmod 755** grants **owner rwx (7)**, **group r-x (5)**, and **others r-x (5)**.

I/O Systems

- 235. I/O techniques are **3** types: **programmed I/O (polling)**, **interrupt-driven I/O**, and **DMA**.
- 236. **Polling (busy-waiting)** wastes CPU by repeatedly checking the device **status/busy bit**.
- 237. **DMA** transfers a whole block **directly between device and memory**, interrupting the CPU **once per block**.
- 238. **DMA** uses **cycle stealing** and frees the CPU from **per-byte** transfer work.
- 239. **Buffering** copes with **speed and size mismatch** between producer and consumer.
- 240. **Spooling** (Simultaneous Peripheral Operation On-Line) buffers output for one-job-at-a-time devices like the **printer**.

Disk Structure and Scheduling

- 241. A disk's physical components are **platters, tracks, sectors**, where a stack of same tracks is a **cylinder**.
- 242. **Disk access time = seek time + rotational latency + transfer time**.
- 243. **Seek time** (moving the arm to a cylinder) is usually the **largest and most-optimized** component.
- 244. Average **rotational latency** equals about **half** a rotation: $(1/2) * (60 / \text{RPM})$ seconds.
- 245. There are **6** disk-scheduling algorithms: **FCFS, SSTF, SCAN, C-SCAN, LOOK, and C-LOOK**.
- 246. **SSTF (Shortest Seek Time First)** serves the **nearest** request next but can cause **starvation**.
- 247. **SCAN (elevator)** moves to **one disk end** servicing requests, then **reverses**.
- 248. **C-SCAN** services in **one direction** only and **jumps back** to the start for a **more uniform** wait time.
- 249. **LOOK and C-LOOK** reverse at the **last request**, not the **physical disk end**, reducing movement.
- 250. For queue **98, 183, 37, 122, 14, 124, 65, 67** with head **53**, **FCFS** total head movement is **640**.
- 251. For that same queue, **SSTF** total head movement is **236**.
- 252. For that queue (moving toward higher cylinders, disk 0-199), **SCAN = 331, C-SCAN = 382, LOOK = 299**, and **C-LOOK = 322**.

RAID and Disk Management

- 253. **RAID** stands for **Redundant Array of Independent Disks** and uses **striping, mirroring, and parity**.
- 254. **RAID 0** uses **striping only** with **no fault tolerance**, needing **2** disks minimum.
- 255. **RAID 1** uses **mirroring** and needs **2** disks minimum.
- 256. **RAID 5** uses **distributed parity** and survives **1** disk failure with **3** disks minimum.
- 257. **RAID 6** uses **double distributed parity** and survives **2** disk failures with **4** disks minimum.
- 258. **RAID 3** uses **byte-level striping with dedicated parity**, while **RAID 4** uses **block-level striping with dedicated parity**.
- 259. Disk preparation has **2** steps: **low-level (physical) formatting** into sectors and **logical formatting** creating the file system.
- 260. The **boot block** holds the **bootstrap** program, often in the **MBR (Master Boot Record)** at **sector 0**.
- 261. **Bad blocks** are handled by **sector sparing (forwarding)** or **sector slipping**.

Bootting, Interrupts and Kernel Internals

- 262. **Bootting** runs **firmware/BIOS POST**, then the **bootstrap loader** loads the **kernel** from disk into memory.
- 263. Interrupts are **3** kinds: **hardware interrupts** (devices), **software interrupts/traps** (system calls), and **exceptions** (faults like divide-by-zero).
- 264. The **interrupt vector** is a table of **ISR (interrupt service routine)** addresses indexed by **interrupt number**.
- 265. A **trap** is a **software-generated interrupt** caused by an **error** or a **user request** for an OS service.

- 266. **Reentrant (pure) code** never modifies itself, so **1** copy can be safely **shared** by many processes.
- 267. **Caching** speeds access using levels such as **registers, L1/L2/L3 cache, main memory, and disk**.
- 268. **POSIX** and **Win32** are the **2** dominant **system-call/API** families on UNIX-like and Windows systems.
- 269. The **memory hierarchy** trades off **speed vs cost vs size**: faster storage (registers) is **smaller and costlier**.

Advanced Process and Thread Concepts

- 270. **fork()** uses **copy-on-write (COW)**, so parent and child **share pages** until one **writes**.
- 271. A **zombie** process has **terminated** but its parent has not called **wait()**, leaving an entry in the **process table**.
- 272. An **orphan** process has a **dead parent** and is **adopted by init (PID 1)** on UNIX.
- 273. A **daemon** is a **background** process, often a **service**, with **no controlling terminal**.
- 274. Common thread libraries are **3: Pthreads (POSIX), Win32 threads, and Java threads**.
- 275. **Pthreads** is a **specification (standard)**, not an implementation.
- 276. A **Lightweight Process (LWP)** is an intermediate structure between **user threads** and **kernel threads**.
- 277. **Thread cancellation** is **2** types: **asynchronous** (terminate immediately) and **deferred** (terminate at **cancellation points**).
- 278. A **thread pool** reuses a fixed number of **pre-created threads**, avoiding per-task **creation overhead**.
- 279. The **first 3** file descriptors are **0 = stdin, 1 = stdout, and 2 = stderr**.

More CPU Scheduling Facts

- 280. **Little's formula** states **$n = \lambda * W$** : average queue length equals **arrival rate** times **average waiting time**.
- 281. With each process idle a fraction **p** for I/O, CPU utilization for **n** processes is **$1 - p^n$** .
- 282. The **convoy effect** in FCFS makes short I/O-bound jobs wait behind **1** long CPU-bound job.
- 283. **SJF/SRTF** minimize average waiting time but require **knowing or predicting** the next CPU burst.
- 284. **Aging** prevents **starvation** by gradually **increasing** the priority of long-waiting processes.
- 285. **Gang scheduling** runs a group of **related threads** simultaneously on **multiple processors**.
- 286. **Real-time** systems prefer **preemptive, priority-based** scheduling to meet **deadlines**.

Advanced Synchronization

- 287. A **spinlock** busy-waits and suits **short** critical sections on **multiprocessors** (no context switch).
- 288. **Priority inversion** occurs when a **high-priority** task waits on a lock held by a **low-priority** task.
- 289. **Priority inheritance** solves priority inversion by temporarily **raising** the low-priority holder's priority.
- 290. A **mutex lock** has **2** operations: **acquire()** and **release()**, and must be held by the **owner** that locked it.
- 291. **Atomic** operations execute as **1** indivisible unit, never **interrupted** midway.
- 292. **Compare-and-Swap (CAS)** underlies **lock-free** data structures and **optimistic** concurrency.
- 293. A **deadlock with semaphores** arises if two processes execute **wait()** on **2** semaphores in **opposite order**.

More Deadlock Facts

- 294. The **4** Coffman conditions are abbreviated as **mutual exclusion, hold-and-wait, no-preemption, and circular wait**.
- 295. **Deadlock detection** frequency is a tradeoff: frequent checks add **overhead**, rare checks delay **recovery**.
- 296. A **wait-for graph** is obtained by **collapsing** resource nodes out of a **single-instance RAG**.
- 297. **Resource preemption** recovery must address **3** issues: **victim selection, rollback, and starvation**.
- 298. **Banker's algorithm** is **conservative**: it may deny a request even when resources are **physically available**.
- 299. A system with a **safe sequence** has **at least 1** ordering in which **all** processes can finish.

Advanced Memory Management

- 300. The **MMU** maps logical to physical addresses using a **relocation (base) register** plus a **limit register** for protection.
- 301. **Overlays** keep only **needed instructions/data** in memory, an old technique for programs **larger than memory**.

- 302. The **buddy system** allocates memory in blocks that are **powers of 2**, splitting and coalescing **buddies**.
- 303. **Slab allocation** caches **kernel objects** of the same type to reduce **fragmentation** and **init cost**.
- 304. The **minimum number of frames** per process is set by the **instruction-set architecture** (an instruction may span pages).
- 305. **Page size** is a tradeoff: **larger pages** reduce table size but increase **internal fragmentation**.
- 306. A **page table base register (PTBR)** points to the page table, avoiding storing the whole table in **registers**.
- 307. **External fragmentation** can reach **1/3** of memory under first-fit (the **50-percent rule** consequence).

More Virtual Memory Facts

- 308. **Copy-on-write** allows parent and child to **share** frames until a **modification** occurs.
- 309. **Memory-mapped files** map a file into the **address space** so file I/O uses **demand paging**.
- 310. **Page buffering** keeps a **pool of free frames** so a fault can be served before writing the **victim**.
- 311. **Demand paging** relies on **locality of reference** to keep the **page-fault rate** low.
- 312. A **page fault** is **not** an error; it is a normal mechanism to load a page **on demand**.
- 313. **Effective access time** worsens sharply because a single fault costs **milliseconds** versus **nanoseconds** for RAM.
- 314. **Local replacement** keeps a process's fault rate **stable**, while **global replacement** can let one process steal frames.

Advanced File Systems

- 315. A file's **type** is often identified by a **magic number** at its start or by its **extension**.
- 316. **Mounting** attaches a file system to the directory tree at a **mount point**.
- 317. A **file descriptor** is an integer index into the **per-process open-file table**.
- 318. The OS keeps **2** open-file tables: a **system-wide** table and a **per-process** table.
- 319. A **hard link** is another **directory entry** to the same **inode**; a **soft (symbolic) link** stores a **path**.
- 320. A **journaling file system** (ext3, ext4, NTFS) **logs** changes before applying them for **crash recovery**.
- 321. Common file systems include **FAT32**, **NTFS (Windows)** and **ext3/ext4**, **XFS (Linux)**.

Advanced Disk and I/O

- 322. A **Solid-State Drive (SSD)** uses **flash memory** with **no seek time or rotational latency**.
- 323. SSDs use **wear leveling** and a **Flash Translation Layer (FTL)** to spread **write cycles**.
- 324. **RAID 10 (1+0)** mirrors then stripes; **RAID 0+1** stripes then mirrors.
- 325. A **device driver** is a **kernel module**, while a **device controller** is the **hardware** managing a device.
- 326. I/O addressing is **2** styles: **memory-mapped I/O** (device registers in the address space) and **port-mapped I/O** (separate I/O space).
- 327. **DMA** has modes including **burst mode** and **cycle-stealing mode**.
- 328. A **disk cache (buffer cache)** holds recently used **disk blocks** in main memory to cut **I/O**.
- 329. The **3** components of disk I/O time are again **seek time**, **rotational latency**, and **transfer time**.
- 330. **Constant Linear Velocity (CLV)** keeps a constant data rate (CDs), while **Constant Angular Velocity (CAV)** keeps constant **RPM** (hard disks).
- 331. **Throughput** of a disk improves when scheduling **minimizes total seek distance**.
- 332. **Interrupt-driven I/O** scales poorly for **high-rate** devices, where **DMA** is preferred.
- 333. A **bootable** disk (system disk) contains the **boot block**, while a **data disk** does not.

High-Yield Numbers and Difference Recap

- 334. The **3** critical-section requirements again: **mutual exclusion**, **progress**, **bounded waiting**.
- 335. The **5** process states again: **new**, **ready**, **running**, **waiting**, **terminated**.
- 336. The **4** deadlock conditions again: **mutual exclusion**, **hold-and-wait**, **no-preemption**, **circular wait**.
- 337. The **3** schedulers again: **long-term (degree of multiprogramming)**, **short-term (fast)**, **medium-term (swapping)**.
- 338. The page-fault EAT formula again: **EAT = (1 - p) * memory access + p * fault service time**.

- 339. **Paging vs segmentation**: paging is **fixed-size + internal fragmentation**; segmentation is **variable-size + external fragmentation**.
- 340. **Preemptive vs non-preemptive**: preemptive can **force** the CPU away; non-preemptive waits for **completion or block**.
- 341. **Process vs thread**: a process has a **separate address space**; threads **share** one within a process.
- 342. **Deadlock vs starvation**: deadlock blocks **all** in the set; starvation lets **others proceed**.
- 343. **Internal vs external fragmentation**: internal wastes space **inside** a block; external scatters free space **outside**.
- 344. **Prevention vs avoidance**: prevention **negates a condition**; avoidance uses **max-need info** to stay **safe**.
- 345. **Logical vs physical address**: logical is **CPU-generated**; physical is **RAM-seen**; the **MMU** maps between them.
- 346. **Mutex vs counting semaphore**: a mutex is **binary (0/1)**; a counting semaphore tracks **N** resource instances.
- 347. **SCAN vs LOOK**: SCAN goes to the **physical disk end**; LOOK reverses at the **last request**.
- 348. **FAT vs inode**: FAT is a **linked-allocation table**; the UNIX **inode** uses a **combined indexed** scheme.
- 349. **Hard real-time vs soft real-time**: hard **guarantees** deadlines; soft only **prioritizes** them.
- 350. **SMP vs AMP**: SMP CPUs are **peers** running the same OS; AMP uses a **master-slave** arrangement.

Specialized Scheduling and IPC

- 351. **Lottery scheduling** gives each process **tickets** and picks a winner **probabilistically**, so shares are proportional to **ticket count**.
- 352. **Fair-share scheduling** allocates CPU among **users or groups** rather than individual **processes**.
- 353. **Remote Procedure Call (RPC)** lets a process call a procedure on a **remote** machine using a **client stub** and **server skeleton**.
- 354. RPC uses **parameter marshalling** to package arguments into a **message** for transmission.
- 355. **Sockets** use **2** transport protocols: **TCP** (connection-oriented, reliable) and **UDP** (connectionless, unreliable).
- 356. A **well-known port** is below **1024**; examples are **HTTP 80**, **HTTPS 443**, and **SSH 22**.
- 357. **Indirect IPC** via a **mailbox** allows **multiple** processes to share **1** communication channel.
- 358. **Blocking (synchronous)** receive makes the receiver **wait** until a message arrives.

System Software: Loaders, Linkers, Compilers

- 359. The **4** language-processing tools are the **compiler**, **assembler**, **linker**, and **loader**.
- 360. A **compiler** translates **high-level language** to **object code**; an **assembler** translates **assembly** to **machine code**.
- 361. A **linker** combines **object modules** and resolves **external references** into one executable.
- 362. A **loader** brings the executable into **memory** and prepares it for **execution**.
- 363. **Static linking** copies library code into each executable; **dynamic linking** shares **1** runtime copy (DLL/.so).
- 364. **Relocation** adjusts addresses so a program can run at a **different** memory location than assumed.

Page-Table Entry and TLB Details

- 365. A **page-table entry (PTE)** holds the **frame number** plus bits: **valid/invalid**, **dirty (modify)**, **reference**, and **protection**.
- 366. The **valid-invalid bit** signals whether a page is **in memory** and **legal** to access.
- 367. The **dirty (modify) bit** marks a page that was **written**, requiring a **write-back** on eviction.
- 368. The **reference bit** records recent **use** and drives **Second-Chance/Clock** and LRU approximations.
- 369. **TLB reach** equals **(number of TLB entries) * (page size)** and bounds how much memory the TLB covers.
- 370. An **ASID (Address-Space Identifier)** in TLB entries avoids a full **TLB flush** on each context switch.
- 371. A **TLB miss** costs an extra **memory access** to read the page table, then updates the TLB.
- 372. **Hierarchical page tables** add **1 memory access per level** unless cached by the **TLB**.

OS History and Examples

- 373. **UNIX** was created in **1969** by **Ken Thompson and Dennis Ritchie** at **Bell Labs**.
- 374. **UNIX** was rewritten in the **C** language around **1973** for portability.

- 375. **Linux** was created in **1991** by **Linus Torvalds** as a **monolithic** kernel.
- 376. **MULTICS** was the influential predecessor that pioneered **segmentation with paging**.
- 377. **MS-DOS** was **single-tasking** and lacked **dual-mode** protection.
- 378. **Windows NT** uses a **hybrid** kernel; **macOS** uses the **XNU (Mach + BSD)** kernel.
- 379. **Android** is built on the **Linux** kernel; **iOS** is built on the **Darwin/XNU** kernel.
- 380. The **init** process has **PID 1**, while the **swapper/scheduler** is **PID 0** on classic UNIX.

More Numerics, Sizes and Edge Cases

- 381. A traditional disk **sector** is **512 bytes**, while **Advanced Format** disks use **4096 bytes (4 KB)**.
- 382. A typical **page/frame size** is **4 KB**, always a **power of 2**.
- 383. **RAID 2** uses **bit-level striping** with **Hamming-code** error correction and is **rarely used**.
- 384. **Context-switch time** is typically a few **microseconds** and is **pure overhead**.
- 385. The **degree of multiprogramming** is the **number of processes** resident in **main memory**.
- 386. **Belady's anomaly** is exhibited by **FIFO** but **never** by **stack algorithms (LRU, OPT)**.
- 387. **Thrashing** begins when CPU utilization suddenly **drops** despite an **increasing** degree of multiprogramming.
- 388. The **working-set window Delta** is the count of recent references defining a process's **working set**.
- 389. A **page fault rate p = 0** means **no faults**, so **EAT** equals the **memory access time**.
- 390. With a **hit ratio** of **100%**, the TLB makes EAT equal to **TLB time + memory time**.
- 391. **FCFS** is the **only** scheduling algorithm that is **purely non-preemptive** and **order-of-arrival** based.
- 392. **Round-Robin** with **n** processes effectively gives each about **1/n** of CPU in slices of **q**.
- 393. The **safe-state** check in Banker's runs in about **O(m * n^2)** time for **m** resources and **n** processes.
- 394. A **resource-allocation graph** with **no cycle** guarantees **no deadlock** in **all** cases.
- 395. **Compaction** is only possible with **execution-time (dynamic)** address binding.
- 396. **First Fit** and **Best Fit** outperform **Worst Fit** in both **speed** and **storage utilization**.
- 397. **Demand segmentation** is used when hardware lacks **paging** support, as in early **OS/2**.
- 398. The **producer-consumer** buffer is **bounded** with **n** slots, tracked by **empty** and **full** semaphores.
- 399. **Dining Philosophers** with the **at-most-4** rule guarantees **at least 1** philosopher can always eat.
- 400. A **monitor** guarantees mutual exclusion **automatically**, unlike **semaphores** which need careful **ordering**.
- 401. **SSTF** improves on **FCFS** but is **not optimal** and may cause **starvation** of far requests.
- 402. **C-LOOK** services requests up to the **highest request**, jumps to the **lowest**, and continues, avoiding the **disk ends**.
- 403. **Seek time** dominates **HDD** access but is **zero** on **SSDs**.
- 404. A **trap/exception** transfers control to the OS through the **interrupt vector**, just like a **hardware interrupt**.
- 405. The **kernel** runs in **kernel mode**, while **user programs** run in **user mode** to enforce **protection**.
- 406. **Multiprogramming** needs **memory protection** and **CPU scheduling** to share resources safely.
- 407. **Spooling** lets the CPU and a **slow device** (printer) overlap by queuing jobs on **disk**.
- 408. A **semaphore** initialized to **0** is used for **signaling/synchronization** between processes.
- 409. **Binary semaphore** and **mutex** both enforce **mutual exclusion**, but a mutex has an **owner**.
- 410. **Page replacement** is needed only on a **page fault** when **no free frame** is available.
- 411. The **optimal** algorithm gives the **lower bound** on page faults and is used as a **benchmark**.
- 412. **LRU** approximates **optimal** by using the **recent past** to predict the **near future**.
- 413. A **dirty page** must be **written to disk** before its frame is **reused**.
- 414. **Frame allocation** must give each process at least its **minimum frames** to make progress.
- 415. The **TLB** is also called an **associative memory** because it searches **all entries in parallel**.
- 416. A **cache hit** in the TLB avoids accessing the **page table** in main memory.
- 417. **Hold-and-wait** is prevented by requiring a process to request **all** resources **before** execution.
- 418. **Circular wait** is prevented by a **global numbering** of resources requested in **increasing** order.
- 419. **No-preemption** is prevented by **forcibly reclaiming** a resource when a request **cannot** be granted.
- 420. **Mutual exclusion** cannot be removed for **inherently non-sharable** resources like a **printer**.

- 421. A process in the **ready** state is in **main memory** waiting only for the **CPU**.
- 422. A process in the **waiting** state needs an **event** (such as **I/O completion**) before becoming **ready**.
- 423. The **dispatcher** must be **fast** because it runs on **every** context switch.
- 424. **Throughput** and **CPU utilization** rise with multiprogramming until **thrashing** sets in.
- 425. A **system call** is **slower** than a normal function call due to the **mode switch** overhead.
- 426. **Kernel-level threads** allow the OS to schedule **threads** on **multiple CPUs** in parallel.
- 427. **Many-to-Many** threading maps **U** user threads onto **K** kernel threads where **$K \leq U$** .
- 428. **Race conditions** are eliminated by enforcing **mutual exclusion** over the **critical section**.
- 429. A **safe sequence** is **not unique**; a state may have **several** valid safe sequences.
- 430. **Deadlock avoidance** keeps the system in a **safe state** at **every** allocation step.
- 431. **Internal fragmentation** in paging is at most **1 page minus 1 byte** per process, in the **last page**.
- 432. **External fragmentation** is solved by **paging** or by **compaction**.
- 433. A **virtual address space** can exceed **physical memory** thanks to **demand paging** and **swapping**.
- 434. **Locality of reference** is **2** types: **temporal** (reuse soon) and **spatial** (nearby addresses).
- 435. **Working-set thrashing** is avoided by ensuring the sum of all working sets is **$\leq$ total frames**.
- 436. **Disk scheduling** matters most for **HDDs** and far **less** for **SSDs** (no seek time).
- 437. The **boot block** is typically at **sector 0**, also known as the **Master Boot Record (MBR)**.
- 438. **RAID levels 0 through 6** trade off **performance, cost, and fault tolerance** differently.